

Algorithm for file updates in Python

Project description

I work as a cybersecurity specialist in a healthcare company, where I'm responsible for controlling access to sensitive systems that handle patient data. To manage this, I regularly maintain a file that lists the IP addresses allowed to access a restricted part of the network.

Access is strictly limited based on these IP addresses, so only approved users can connect. Alongside this allow list, I also work with a separate remove list, which contains IP addresses that should no longer have access—usually because an employee's role has changed or their permissions have been revoked.

My task is to develop a Python-based solution that checks whether any IP addresses from the remove list are still present in the allow list. If such matches are found, the program should automatically remove those IPs from the allow list to keep access permissions accurate and secure.

To make things more realistic, I created a function with two parameters: the main list with all IP addresses (`import_file`) and the list to remove (`remove_list`). The whole algorithm was written within that function.

```
def update_list(import_file, remove_list):
```

Open the files that contain the two lists

I used a `with` statement to open the files. In my algorithm, the `with` statement works together with the `open()` function in read mode to access the files of lists. This allows me to read the IP addresses stored in those files. The `with` keyword is important because it automatically manages system resources by closing the file once the block of code is finished.

(Sometimes I will attach two screenshots, because I worked with two lists, but describe only one, if the logic of the lines is the same)

```
with open(import_file, "r") as file:
```

```
with open(remove_list, "r") as file:
```

In the lines `with open(import_file, "r") as file:`, the `open()` function takes two arguments. The first specifies the file I want to open, and the second defines the mode in which I want to use it. In this case, `"r"` stands for read mode. The `as` keyword assigns the opened file to a variable named `file`, which I can then use to work with the file's contents inside the `with` block.

Read the contents of the files

To read the contents of the files, I used the `.read()` method, which converts the data into a string. When the `open()` function is used with the `"r"` (read) mode, the `.read()` method can be called inside the `with` block. This method allows me to access the full content of the file as a single string. I applied `.read()` to the file object created in the `with` statement and stored the result in the variable `ip_addresses`.

In summary, this step loads the contents of the `"import_file"` into a string format, which can later be processed and analyzed in the program.

```
ip_addresses = file.read()
```

```
ip_deleted = file.read()
```

Convert the strings into lists

To work with individual IP addresses, I needed to convert the string into a list. For this purpose, I used the `.split()` method. This method transforms a string into a list by separating its elements based on a delimiter. By default, `.split()` uses whitespace as the separator.

In this case, the string stored in `ip_addresses` contained IP addresses separated by spaces, so `.split()` converted it into a list where each IP address became a separate element. I then reassigned this list back to the variable `ip_addresses` for further processing.

```
ip_addresses = ip_addresses.split()
```

```
ip_deleted = ip_deleted.split()
```

Iterate through the main list

An important part of the algorithm is iterating through the list of IP addresses. To achieve this, I used a `for` loop.

A `for` loop allows repeated execution of code for each element in a sequence. In this case, the loop goes through each IP address and assigns it to the variable `element`. This makes it possible to apply specific operations to every item in the list.

```
for element in ip_addresses:
```

Remove IP addresses that are in the remove list

The goal of the algorithm is to remove any IP address from the allow list (`ip_addresses`) if it is also present in the remove list (`ip_deleted`).

Inside the loop, I used a conditional statement to check whether the current element exists in the list of IP addresses. This step helps prevent errors that could occur if trying to remove an element that is not present.

If the condition is satisfied, I used the `.remove()` method to delete that IP address from `ip_addresses`. This ensures that any address found in the remove list is excluded from the final allow list.

```
if element in ip_deleted:  
    ip_addresses.remove(element)
```

Update the file with the revised list of IP addresses

At one of the final stages of my algorithm, I needed to update the allow list file with the modified set of IP addresses. To do this, I first converted the list back into a string using the `.join()` method. This method merges all elements of an iterable into a single string, using a specified separator. In my case, I applied `.join()` to the `ip_addresses` list and used `"; "` as the separator so that each IP address would be clearly separated in the resulting string. This allowed me to pass the formatted data into the `.write()` method.

Next, I used another with statement along with the `.write()` method to update the file. This time, I opened the file in write mode by passing `"w"` as the second argument to the `open()` function. This mode allows me to overwrite the existing contents of the file. Inside

the with block, I called the .write() method, which writes string data to the file and replaces any previous content.

In this step, I wrote the updated list of IP addresses back to "import_file". As a result, any IP addresses that were removed from the allow list no longer have access to the restricted system.

```
ip_addresses = "; ".join(ip_addresses)

with open (import_file, "w") as file:
    file.write(ip_addresses)
```

Check the results in Python

To verify the outcome, I reopened the file and read its contents again. I stored the result in a variable named upd and returned it from the function. Finally, I called the function with the appropriate file names to confirm that the updates were applied correctly.

```
with open (import_file, "r") as file:
    upd = file.read()

return upd

print(update_list("allow_list.txt", "delete_list.txt"))

192.168.145.158; 192.168.108.13; 192.168.60.153; 192.168.96.200; 192.168.247.153
Press any key to continue . . .
```

Summary

I created an algorithm within a function that removes IP addresses identified in a ip_deleted list from the "import_file" file of approved IP addresses. This algorithm involved opening the file, converting it to a string to be read, and then converting this string to a list stored in the variable ip_addresses. I then iterated through the IP addresses in ip_deleted. With each iteration, I evaluated if the element was part of the ip_addresses list. If it was, I applied the .remove() method to it to remove the element from ip_addresses.. After this, I used the .join() method to convert the ip_addresses back into a string so that I could write over the contents of the "import_file" file with the revised list of IP addresses.